TP10 : Tas-Min et tri par tas en Python

CPES « Sciences des données, arts et cultures » : Introduction à l'algorithmique

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

02 avril 2024

Avant de commencer le TP10, nous allons finir la correction des exercices « sur papier » du TD/TP9.

Tas-Min : compréhension et implémentation en Python

Dans cet exercice, on se propose d'étudier un autre tas binaire que celui vu en cours : le Tas-Min, complémentaire du Tas-Max.

Question 1 Afin de bien comprendre les procédures écrites dans la question précédente, étudions un cas particulier « sur papier ». On considère le tableau suivant :
`[16, 14, 10, 9, 8, 7, 4, 3, 2, 1]`. Commencer par écrire ce tableau sous la forme d'un arbre binaire complet comme vu en cours dans la slide 9. En se basant sur les slides 14 et 15 du cours, écrire ensuite l'état de l'arbre binaire après chaque appel de la procédure ENTASSER-MIN au sein d'un appel de CONSTRUCTION-TAS-MIN sur le tableau `[16, 14, 10, 9, 8, 7, 4, 3, 2, 1]`. Pour rappel, de façon analogue au Tas-Max vu en cours, la deuxième procédure appelle la première plusieurs fois afin de transformer un tableau d'entiers passé en paramètre en un arbre binaire complet (sauf éventuellement sur le dernier niveau), **où la clé de chaque sommet est plus petite que la clé de ses enfants**.

Question 2 Ecrire deux fonctions Python `gauche` et `droite`, prenant en paramètre un indice de l'arbre binaire complet `i`, et retournant respectivement l'indice de son fils gauche et de son fils droit, si jamais ils existaient. **Attention : Dans le cours, GAUCHE(i) et DROITE(i) retournent des indices en partant du principe que le tableau qui représente l'arbre binaire est indexé à partir de 1. Ce n'est pas le cas en Python : les listes Python sont indexées à partir de 0!** Il faut donc adapter les indices en conséquence.

Question 3 Écrire une fonction Python `entasser_min` prenant en paramètre une liste Python d'entiers `A` représentant un arbre binaire complet, et `i` un entier représentant l'indice du noeud de l'arbre que l'on veut replacer pour respecter le Tas-Min. Cette fonction devra retourner la liste Python d'entiers `A` modifiée, donc où la valeur à l'indice `i` a été replacée.

Question 4 Écrire une fonction Python `construction_tas_min` prenant en paramètre une liste Python d'entiers `A` que l'on veut transformer en tas binaire Tas-Min. Cette fonction devra donc retourner la liste Python d'entiers `A` représentant maintenant un Tas-Min.

Question 5 Exécuter la fonction `construction_tas_min` sur `A = [16, 14, 10, 9, 8, 7, 4, 3, 2, 1]`. Vérifier que le résultat obtenu correspond à ce que vous aviez « sur papier » dans la première question.

Tri par tas décroissant

Dans ce second exercice, on se propose d'utiliser notre implémentation de Tas-Min afin d'écrire un algorithme de tri décroissant, que l'on appellera tri par tas décroissant, qui offre une complexité temporelle en $O(n\log(n))$.

Comme l'indique le pseudo-code du tri par tas à la slide 18 du cours, il y a deux notions de taille à prendre en compte dans le tri. La taille réelle du tableau `A`, qui est fixe au cours du temps ; et la taille du sous-tableau de `A` qu'il reste à trier. En effet, le tri par tas croissant/décroissant va, à la première itération de la boucle pour, faire remonter la plus grande/plus petite valeur du tableau `A` dans sa première case, puis l'échanger avec la valeur de la dernière case. La dernière case du tableau contenant maintenant la plus grande/plus petite valeur, il ne nous reste plus qu'à répéter l'opération sur le tableau `A` sans sa dernière case. On procède ainsi de suite jusqu'à ce qu'il n'y ait plus qu'une seule case à traiter, et on obtiendra ainsi le tableau trié par ordre croissant/décroissant. On aura alors bien fait n appels à ENTASSER-MAX/ENTASSER-MIN dont la complexité est en $O(\log(n))$, qui donne bien une complexité totale en $O(n\log(n))$.

Question 6 La fonction Python `construction_tas_min` permet d'appliquer la procédure ENTASSER-MIN en considérant l'ensemble du tableau `A` passé en paramètre. Toutefois, on a besoin de pouvoir appliquer cette procédure uniquement sur une sous-partie du tableau, et plus particulièrement, sur le sous-tableau consistué des n premières cases du tableau. Écrire alors une fonction Python `entasser_min_n` prenant les mêmes paramètres `A` et `i` que `entasser_min`, ainsi qu'un paramètre `n` représentant la valeur de n décrite plus tôt.

Question 7 En vous inspirant de la slide 18 du cours, écrire une fonction Python `tri_par_tas_decroissant` prenant en entrée une liste Python d'entiers `A` et retournant cette liste d'entiers triée par ordre décroissant, en exploitant les fonctions Python déjà écrites `construction_tas_min` et `entasser_min_n`.

Question 8 Vérifier que l'algorithme de tri décroissant implémenté est correct en l'utilisant sur le tableau d'entiers `A = [16, 14, 10, 9, 8, 7, 4, 3, 2, 1]`.